

Attention Is All You Need - DL 1팀 리뷰

[1706.03762] Attention Is All You Need

Abstract

왜 새로운 모델이 필요했을까?

- Transformer가 나오기 전에는 RNN이나 LSTM을 사용했었음.
- **순차적 연산 (Sequential Processing):** 단어를 하나씩 순서대로 읽어야 합니다. 병렬 처리가 불가능해 학습 속도가 매우 느렸습니다.
- **장기 의존성 (Long-term Dependency) 문제:** 문장이 길어지면 앞부분에 나온 단어의 정보를 잊어버리는 경향이 있었습니다.
- **Transformer의 제안:** 순환과 합성곱을 완전히 배제하고, 오로지 어텐션 메커니즘에만 의존하는 아주 단순한 네트워크 구조를 제안합니다. 이러한 과정을 병렬적으로 처리하기 때문에 RNN보다 성능과 속도적인 부분에서 훨씬 뛰어났습니다.

Introduction

- **RNN의 한계:** 기존 언어 모델링이나 기계 번역에서는 RNN, LSTM, GRU가 확실한 최고 성능 모델로 자리 잡고 있었습니다. 하지만 RNN은 입력과 출력의 기호 위치에 따라 연산을 순차적으로 수행합니다
- **어텐션의 등장과 한계:** 어텐션 메커니즘은 입력과 출력 시퀀스의 거리에 상관없이 의존성을 모델링할 수 있게 해 주어 강력한 모델의 필수적인 부분이 되었습니다. 하지만 여전히 예외적인 몇 경우를 제외하고는 대부분 순환 신경망(RNN)과 함께 사용되고 있었습니다.
- 본 논문에서는 순환(recurrence)을 피하고 어텐션 메커니즘에만 전적으로 의존하여 입력과 출력 간의 전역적인 의존성(global dependencies)을 이끌어내는 **Transformer**를 제안합니다.

Background

병렬 처리를 시도했던 CNN 기반 모델들도 먼 거리의 단어 관계를 파악하는 데는 한계가 있었습니다.

- **CNN 모델들과의 비교:** 순차적 연산을 줄이기 위해 합성곱 신경망(CNN)을 기본 블록으로 사용하는 ByteNet이나 ConvS2S 같은 모델들이 있었습니다. 하지만 이 모델들은 두 위치(단어) 사이의 거리가 멀어질수록 필요한 연산량이 선형적(ConvS2S) 혹은 로그 형태(ByteNet)로 증가합니다. 이로 인해 먼 거리에 있는 단어들 간의 의존성을 학습하기 어렵다는 단점이 있었습니다.
- **Transformer의 우수성:** 트랜스포머는 이를 상수 번의 연산으로 줄였습니다. 즉, 문장 내에서 단어 사이의 거리가 아무리 멀어도 한 번에 관계를 파악할 수 있습니다.
- **Self-attention (자기 주의 집중):** 단일 시퀀스 내의 서로 다른 위치들을 연결하여 그 시퀀스의 표현(representation)을 계산하는 어텐션 메커니즘입니다. 트랜스포머는 시퀀스에 맞춰진 RNN이나 합성곱을 전혀 사용하지 않고, 오직 Self-attention에만 의존하여 입력과 출력의 표현을 계산하는 최초의 변환 모델입니다.

Model Architecture

트랜스포머는 기존의 가장 경쟁력 있는 신경망 시퀀스 변환 모델들이 따르던 **인코더-디코더 (Encoder-Decoder)** 구조를 기반으로 합니다.

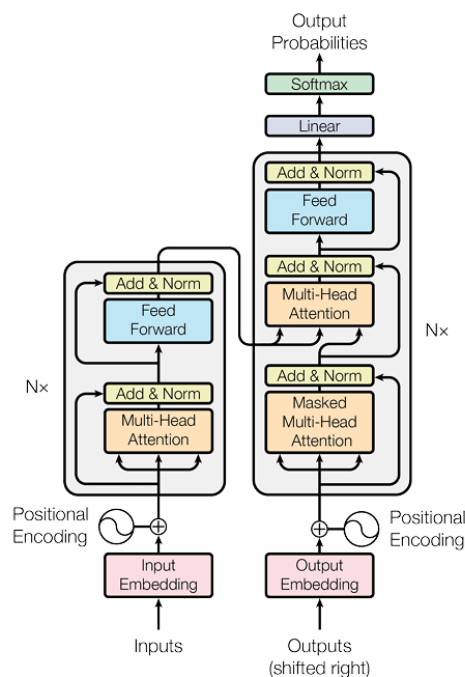


Figure 1: The Transformer - model architecture.

- **인코더 (Encoder)**

인코더는 N=6개의 완전히 동일한 레이어가 쌓여 있는 구조입니다. 각 레이어 안에는 두 개의 하위 레이어(sub-layer)가 존재합니다.

Multi-Head Self-Attention 메커니즘

Position-wise Fully Connected Feed-Forward Network

잔차 연결 및 정규화: 각 하위 레이어의 출력은 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 형태입니다. 즉, 입력을 그대로 더해주는 잔차 연결(Residual Connection)을 적용한 뒤, 층 정규화(Layer Normalization)를 수행합니다.

이를 위해 모델 내의 모든 하위 레이어와 임베딩 레이어는 $d=512$ 라는 고정된 출력 차원을 가집니다.

- **디코더 (Decoder)**

디코더 역시 N=6개의 동일한 레이어가 쌓여 있습니다. 인코더와 비슷하지만 결정적인 차이점들이 있습니다.

세 번째 하위 레이어 추가: 인코더의 스택 출력 결과에 대해 multi-head attention을 수행하는 층이 하나 더 있습니다. 디코더 역시 각 하위 레이어 주변에 잔차 연결과 층 정규화를 적용합니다.

마스킹(Masking) 기법: 디코더 내부의 self-attention 하위 레이어는 미래의 정보를 미리 보지 못하도록 마스킹(Masking) 처리를 합니다.

- **Attention**

어텐션 함수는 하나의 Query(질의)와 Key-Value(키-값) 쌍들을 Output(출력)으로 매핑하는 과정입니다. 출력은 Value들의 가중합(weighted sum)으로 계산됩니다.

1. Scaled Dot-Product Attention

논문에서 제안한 특별한 어텐션 방식입니다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

작동 방식: Query와 모든 Key의 내적(dot product)을 구합니다. (얼마나 연관성이 있는지 점수를 매기는 과정입니다.) 이후 스케일링을 진행합니다. (vanishing gradient 발생을 막기 위해서)

그리고 **Softmax 적용합니다.** 스케일링된 값에 softmax를 씌워 0~1 사이의 가중치 확률값을 얻고, 이를 Value에 곱합니다.

2. Multi-Head Attention

단일 어텐션을 사용하는 대신, 연산을 여러 번 나누어 병렬로 수행하는 기법입니다.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

Multi-head 어텐션을 사용하면 모델이 각기 다른 위치의 다양한 표현 부분 공간에서 나오는 정보에 동시에 집중할 수 있습니다. (한 헤드는 주어-동사 관계를 보고, 다른 헤드는 형용사-명사 관계를 보는 식입니다.)

3. Position-wise Feed-Forward Networks

어텐션 하위 레이어를 거친 후, 인코더와 디코더의 각 계층은 완전히 연결된 피드포워드 네트워크(FFN)를 통과합니다. 이 연산은 각 위치마다 개별적이고 동일하게 적용됩니다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

두 번의 선형 변환(Linear transformation)을 거치며, 그 사이에 ReLU 활성화 함수가 들어갑니다.

입력과 출력의 차원은 $d=512$ 이고, 내부의 은닉층 차원은 훨씬 큰 $d=2048$ 입니다.

4. Positional Encoding

모델이 시퀀스의 순서 정보를 사용할 수 있도록, 인코더와 디코더 맨 하단의 입력 임베딩에 **Positional encodings** 값을 더해줍니다. 더하기 연산이 가능하도록 임베딩과 동일한 차원을

가집니다. 본 논문에서는 주파수가 다른 **사인(sine)**과 **코사인(cosine)** 함수를 사용해 이 위치 값을 생성합니다.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Why Self-Attention

기존의 RNN이나 CNN과 비교했을 때, Self-Attention은 **계산 복잡도**, **병렬 처리 능력**, **장기 의존성 학습**이라는 세 가지 측면에서 압도적인 우위를 가집니다.

- 레이어당 총 계산 복잡도 (Complexity per Layer)
- 병렬화 가능한 연산량 (Sequential Operations): 순차적으로 실행해야 하는 최소 연산 횟수입니다.
- 네트워크 내 장거리 의존성 간의 경로 길이 (Maximum Path Length): 문장 내의 단어들이 서로 얼마나 멀리 떨어져 있든, 정보를 교환하기 위해 거쳐야 하는 경로의 길이입니다. 경로가 짧을수록 먼 거리의 단어 관계를 학습하기 쉽습니다.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional (CNN)	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

Training

대규모 데이터셋과 8개의 GPU를 사용해 학습했으며, 기존 모델 대비 학습 시간이 혁신적으로 단축되었습니다.

Optimizer

- **Training Data**

영어-독일어(EN-DE): 약 450만 개의 문장 쌍으로 구성된 WMT 2014 데이터셋을 사용했습니다. (약 37,000개의 Byte-Pair Encoding 토큰 사용)

영어-프랑스어(EN-FR): 훨씬 더 큰 3,600만 개의 문장 쌍으로 구성된 WMT 2014 데이터셋을 사용했습니다.

- **Optimizer**

Adam 옵티마이저를 사용. 학습 초반에는 학습률(Learning Rate)을 선형적으로 증가시키다가, 일정 STEP(warmup_steps=4000) 이후에는 스텝 수의 역제곱근에 비례하여 감소시키는 특수한 방식을 사용함.

- **Regularization**

Dropout = 0.1

Label Smoothing = 0.1

Results

번역 품질에서 기존의 모든 기록을 갈아치웠으며(SOTA 달성), 번역 외의 다른 자연어 처리 작업(구문 분석)에서도 훌륭한 성능을 입증했습니다.

- **Machine Translation SOTA 달성**

영어-독일어: Big 모델이 28.4 BLEU 스코어를 기록하며, 기존 최고 성능의 앙상블 모델들보다 2.0 BLEU 이상 앞서는 새로운 SOTA(State-of-the-Art)를 달성했습니다.

영어-프랑스어: Big 모델이 41.8 BLEU 스코어를 기록하며 기존 최고 단일 모델들을 뛰어넘었습니다.

- **Model Variations**

어텐션 헤드의 수, 차원 크기 등을 조절해 본 결과, 모델이 커질수록 성능이 좋아지며 드롭아웃이 과적합 방지에 매우 효과적임을 확인했습니다.

- **Generalization**

이 모델이 번역에만 특화된 것인지 확인하기 위해 영어 구문 분석(Constituency Parsing) 작업에도 테스트를 진행했습니다. 작업에 특화된 튜닝이 부족했음에도 불구하고

매우 훌륭한 결과를 내어, 다른 자연어 처리 작업에도 훌륭하게 일반화될 수 있음을 증명했습니다.

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

- 학습 비용 최소화